МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ

КИЇВСЬКИЙ ПОЛІТЕХНІЧНИЙ ІНСТИТУТ

імені ІГОРЯ СІКОРСЬКОГО

НАЦІОНАЛЬНИЙ ТЕХНІЧНИЙ УНІВЕРСИТЕТ УКРАЇНИ

Комп'ютерний практикум №4

з дисципліни “Криптографія”

Тема:

# Вивчення криптосистеми RSA та алгоритму електронного підпису; ознайомлення з методами генерації параметрів для асиметричних криптосистем

Виконав:

Бойко Євгеній ФБ-42

Сушко Ілля ФБ-43

Київ - 2026

**Мета:** ознайомлення з тестами перевірки чисел на простоту і методами генерації ключів для асиметричної криптосистеми типу RSA; практичне ознайомлення з системою захисту інформації на основі криптосхеми RSA, організація з використанням цієї системи засекреченого зв'язку й електронного підпису, вивчення протоколу розсилання ключів.

---

# Хід роботи

Виходячи із мети було написано наступні коди модулі:

- `gen_prime_rand` - генератор випадкових чисел, що потім перевіряються за допомогою тесту Міллера-Рабіна
- `gen_prime_pair` - використовує попередній модуль і з отриманих простих чисел $p', q'$ генерує пару простих $p = 2ip' + 1$, $i = 1, 2, 3, ..., n$ та $q = 2iq' + 1$
- `gen_RSA_pair` - зі згенерованої пари чисел створює публічний та приватний ключі $(e, n)$, $(d, p, q)$ відповідно, де $n = pq$, $d \equiv e^{-1} \mod \phi(n)$, $e = 2^{16} + 1$
- `rsa` - реалізовує всі потрібні примітиви
- `protocol` - реалізує протокол відправки ключа наведений у методичних вказівках
- `site_interop` - напівавтоматичний спосіб комунікації із тестовим сайтом

Таким чином було знайдено прості числа, що задовольняють умову стійкості до криптоатак на RSA

```
Знайдено просте число p' =
506003760246191534467577932298334831428723895913492337896747160594127861295l3
у діапазоні [255, 255] біт.
p = 2 * 1 * p' + 1 =
101200752049238306893515586459666966285744779182698467579349432118825572259027
не є простим. Продовжуємо пошук...
p = 2 * 2 * p' + 1 =
202401504098476613787031172919333932571489558365396935158698864237651144518053
не є простим. Продовжуємо пошук...
p = 2 * 3 * p' + 1 =
303602256147714920680546759379000898857234337548095402738048296356476716777079
не є простим. Продовжуємо пошук...

<SNIP>

p = 2 * 196 * p' + 1 =
198353474016507081511290549460947253920059767198088996455524886952898121627690
97 не є простим. Продовжуємо пошук...
p = 2 * 197 * p' + 1 =
```

```
199365481536999464580225705325543923582917214989915981131318381274086377350281
23 не є простим. Продовжуємо пошук...
Знайдено стійке просте число p =
200377489057491847649160861190140593245774662781742965807111875595274633072871
49 (2 * 198 * p' + 1).




Знайдено просте число p' =
578434279512872476327962697399393307725169877707833942109624317430535533327 17
у діапазоні [255, 255] біт.
p = 2 * 1 * p' + 1 =
115686855902574495265592539479878661545033975541566788421924863486107106665435
не є простим. Продовжуємо пошук...
p = 2 * 2 * p' + 1 =
231373711805148990531185078959757323090067951083133576843849726972214213330869
не є простим. Продовжуємо пошук...
p = 2 * 3 * p' + 1 =
347060567707723485796777618439635984635101926624700365265774590458321319996303
не є простим. Продовжуємо пошук...

<SNIP>

p = 2 * 234 * p' + 1 =
270707242812024318921486542382916068015379502767266284907304180557490629597115
57 не є простим. Продовжуємо пошук...
p = 2 * 235 * p' + 1 =
271864111371050063874142467777714854630829842522681952791523429192351700663769
91 не є простим. Продовжуємо пошук...
p = 2 * 236 * p' + 1 =
273020979930075808826798393172513641246280182278097620675742677827212771730424
25 не є простим. Продовжуємо пошук...
Знайдено стійке просте число p =
274177848489101553779454318567312427861730522033513288559961926462073842797078
59 (2 * 237 * p' + 1).







Знайдено просте число p' =
301022737796666242560074519072896609142928088256575530707997601459747259152 33
у діапазоні [255, 255] біт.
```

```
p = 2 * 1 * p' + 1 =
602045475593332485120149038145793218285856176513151061415995202919494518304 67
не є простим. Продовжуємо пошук...

<SNIP>

p = 2 * 92 * p' + 1 =
553881837545865886310537115094129760822987682392098976502715586685934956840287
3 не є простим. Продовжуємо пошук...
p = 2 * 93 * p' + 1 =
559902292301799211161738605475587693005846244157230487116875538715129902023333
9 не є простим. Продовжуємо пошук...
p = 2 * 94 * p' + 1 =
565922747057732536012940095857045625188704805922361997731035490744324847206380
5 не є простим. Продовжуємо пошук...
Знайдено стійке просте число p =
571943201813665860864141586238503557371563367687493508345195442773519792389427
1 (2 * 95 * p' + 1).




Знайдено просте число p' =
410844972194771913621234096090760358118150127487291821546766380242733629838 39
у діапазоні [255, 255] біт.
p = 2 * 1 * p' + 1 =
821689944389543827242468192181520716236300254974583643093532760485467259676 79
не є простим. Продовжуємо пошук...
p = 2 * 2 * p' + 1 =
164337988877908765448493638436304143247260050994916728618706552097093451935357
не є простим. Продовжуємо пошук...
p = 2 * 3 * p' + 1 =
246506983316863148172740457654456214870890076492375092928059828145640177903035
не є простим. Продовжуємо пошук...
p = 2 * 4 * p' + 1 =
328675977755817530896987276872608286494520101989833457237413104194186903870713
не є простим. Продовжуємо пошук...
p = 2 * 5 * p' + 1 =
410844972194771913621234096090760358118150127487291821546766380242733629838391
не є простим. Продовжуємо пошук...
p = 2 * 6 * p' + 1 =
493013966633726296345480915308912429741780152984750185856119656291280355806069
не є простим. Продовжуємо пошук...
p = 2 * 7 * p' + 1 =
575182961072680679069727734527064501365410178482208550165472932339827081773747
```

```
не є простим. Продовжуємо пошук...
p = 2 * 8 * p' + 1 =
65735195551163506179397455374521657298904020397966691447482620838837380774142 5
не є простим. Продовжуємо пошук...
p = 2 * 9 * p' + 1 =
73952094995058944451822137296336864461267022947712527878417948443692053370910 3
не є простим. Продовжуємо пошук...
p = 2 * 10 * p' + 1 =
82168994438954382724246819218152071623630025497458364309353276048546725967678 1
не є простим. Продовжуємо пошук...
p = 2 * 11 * p' + 1 =
90385893882849820996671501139967278785993028047204200740288603653401398564445 9
не є простим. Продовжуємо пошук...
Знайдено стійке просте число p =
98602793326745259269096183061782485948356030596950037171223931258256071161213 7
(2 * 12 * p' + 1).


((57194320181366586086414158623850355737156336768749350834519544277351979238942 71,
98602793326745259269096183061782485948356030596950037171223931258256071161213 7
),
(20037748905749184764916086119014059324577466278174296580711187559527463307287 149,
27417784848910155377945431856731242786173052203351328855996192646207384279707859))
```

Ітерації роботи алгоритму Мілера-Рабіна для одного із чисел-генераторів ключа:

```
Тестуємо
57194320181366586086414158623850355737156336768749350834519544277351979238942 71 на простоту з 10 ітераціями.
Розкладаємо
57194320181366586086414158623850355737156336768749350834519544277351979238942 70 як 2^1 *
28597160090683293043207079311925177868578168384374675417259772138675989619471 35

--- Ітерація 1 ---
Випадкова основа a:
12832097809889401312570422653513501312243702696287280228059215332863091858141 09
Обчислюємо
12832097809889401312570422653513501312243702696287280228059215332863091858141 09^28597160090683293043207079311925177868578168384374675417259772138675989619471 35 mod
```

```
571943201813665860864141586238503557371563367687493508345195442773519792389427
1 =
571943201813665860864141586238503557371563367687493508345195442773519792389427
0
571943201813665860864141586238503557371563367687493508345195442773519792389427
1 може бути простим (x =
571943201813665860864141586238503557371563367687493508345195442773519792389427
0). Переходимо до наступної ітерації.

--- Ітерація 2 ---
Випадкова основа a:
363994319042111724012305911031600273201659254611583838901912568555216719886307
4
Обчислюємо
363994319042111724012305911031600273201659254611583838901912568555216719886307
4^285971600906832930432070793119251778685781683843746754172597721386759896194 7
135 mod
571943201813665860864141586238503557371563367687493508345195442773519792389427
1 = 1
571943201813665860864141586238503557371563367687493508345195442773519792389427
1 може бути простим (x = 1). Переходимо до наступної ітерації.

--- Ітерація 3 ---
Випадкова основа a:
271325440384836445601678080506844806789944972840005793815314743577111747646732
6
Обчислюємо
271325440384836445601678080506844806789944972840005793815314743577111747646732
6^285971600906832930432070793119251778685781683843746754172597721386759896194 7
135 mod
571943201813665860864141586238503557371563367687493508345195442773519792389427
1 =
571943201813665860864141586238503557371563367687493508345195442773519792389427
0
571943201813665860864141586238503557371563367687493508345195442773519792389427
1 може бути простим (x =
571943201813665860864141586238503557371563367687493508345195442773519792389427
0). Переходимо до наступної ітерації.

--- Ітерація 4 ---
Випадкова основа a:
428594912450479068718168311158810287152967690322982343646847963722865465506422
Обчислюємо
428594912450479068718168311158810287152967690322982343646847963722865465506422
^285971600906832930432070793119251778685781683843746754172597721386759896194 71
35 mod
```

```
571943201813665860864141586238503557371563367687493508345195442773519792389427
1 = 1
571943201813665860864141586238503557371563367687493508345195442773519792389427
1 може бути простим (x = 1). Переходимо до наступної ітерації.

--- Ітерація 5 ---
Випадкова основа a:
545743767866182131170120321160466467538703177486120727809992709386812853956265
6
Обчислюємо
545743767866182131170120321160466467538703177486120727809992709386812853956265
6^285971600906832930432070793119251778685781683843746754172597721386759896194 7
135 mod
571943201813665860864141586238503557371563367687493508345195442773519792389427
1 = 1
571943201813665860864141586238503557371563367687493508345195442773519792389427
1 може бути простим (x = 1). Переходимо до наступної ітерації.

--- Ітерація 6 ---
Випадкова основа a:
495966633000577172534449381661261201964325984059690885657531530834832013115560
8
Обчислюємо
495966633000577172534449381661261201964325984059690885657531530834832013115560
8^285971600906832930432070793119251778685781683843746754172597721386759896194 7
135 mod
571943201813665860864141586238503557371563367687493508345195442773519792389427
1 =
571943201813665860864141586238503557371563367687493508345195442773519792389427
0
571943201813665860864141586238503557371563367687493508345195442773519792389427
1 може бути простим (x =
571943201813665860864141586238503557371563367687493508345195442773519792389427
0). Переходимо до наступної ітерації.

--- Ітерація 7 ---
Випадкова основа a:
427623650297905926490837500766610203937295204953613622456579534281872150547875
3
Обчислюємо
427623650297905926490837500766610203937295204953613622456579534281872150547875
3^285971600906832930432070793119251778685781683843746754172597721386759896194 7
135 mod
571943201813665860864141586238503557371563367687493508345195442773519792389427
1 = 1
571943201813665860864141586238503557371563367687493508345195442773519792389427
```

```
1 може бути простим (x = 1). Переходимо до наступної ітерації.

--- Ітерація 8 ---
Випадкова основа a:
10545181590289036465162317032334448186169187752903470633128045822096682635860
99
Обчислюємо
10545181590289036465162317032334448186169187752903470633128045822096682635860
99^285971600906832930432070793119251778685781683843746754172597721386759896194
135 mod
57194320181366586086414158623850355737156336768749350834519544277351979238942
71 = 1
57194320181366586086414158623850355737156336768749350834519544277351979238942
71 може бути простим (x = 1). Переходимо до наступної ітерації.

--- Ітерація 9 ---
Випадкова основа a:
15145909511731094393526580133686454546767207753706233839750580005851787203932
51
Обчислюємо
15145909511731094393526580133686454546767207753706233839750580005851787203932
51^285971600906832930432070793119251778685781683843746754172597721386759896194
135 mod
57194320181366586086414158623850355737156336768749350834519544277351979238942
71 = 1

57194320181366586086414158623850355737156336768749350834519544277351979238942
71 може бути простим (x = 1). Переходимо до наступної ітерації.

--- Ітерація 10 ---
Випадкова основа a:
24175113102665557973271041523764775196619270099431387159617095930450996518698
79
Обчислюємо
24175113102665557973271041523764775196619270099431387159617095930450996518698
79^285971600906832930432070793119251778685781683843746754172597721386759896194
135 mod
57194320181366586086414158623850355737156336768749350834519544277351979238942
71 = 1

57194320181366586086414158623850355737156336768749350834519544277351979238942
71 може бути простим (x = 1). Переходимо до наступної ітерації.

Всі 10 ітерацій пройдено.
57194320181366586086414158623850355737156336768749350834519544277351979238942
71
```

```
1 з високою ймовірністю є простим.
True
```

Генерація пари ключів

```
PS > python gen_RSA_pair.py
Пара ключів A:
((507089519363288393475057863582198312333457510767089438236736347511876528197973707099196337074933985451491463170374706353384174055285898345098052576660219371
3,
571943201813665860864141586238503557371563367687493508345195442773519792389427
1,
986027933267452592690961830617824859483560305969500371712239312582560711612137
),
(563951973230698492137570079342709545307027183273816635505794939790015985822101891373771453311049685827061431864669244850785862971286898758166325412554836712727, 65537))
d * e mod phi(n) = 1
Пара ключів B:
((412464257428588618137777650733636932312566041295094662635461291300334004621756349038841426560331112883460852331503953443431611925239459730830832459229019972169,
200377489057491847649160861190140593245774662781742965807111875595274633072871749,
274177848489101553779454318567312427861730522033513288559961926462073842797078559),
(549390688354316043064356520865198537344646477823641219135829657702782140538098249739104890006461662790046193115173350244496034738730704117486505490563545003991, 65537))
d * e mod phi(n) = 1

Перевірка співвідношень для ключів абонента A (довжина n: 521 біт, p: 262 біт, q: 260 біт):
  p, q - прості числа (Міллер-Рабін): True
  p != q: True
  n = p * q: True
  gcd(e, phi(n)) = 1: True
  2 <= e <= phi(n) - 1: True
  d * e = 1 (mod phi(n)): True
  0 < d < phi(n): True
  (M^e)^d mod n = M для випадкового M: True

Перевірка співвідношень для ключів абонента B (довжина n: 528 біт, p: 264 біт, q: 264 біт):
  p, q - прості числа (Міллер-Рабін): True
  p != q: True
```

```
  n = p * q: True
  gcd(e, phi(n)) = 1: True
  2 <= e <= phi(n) - 1: True
  d * e = 1 (mod phi(n)): True
  0 < d < phi(n): True
  (M^e)^d mod n = M для випадкового M: True

Умова p*q <= p1*q1 (n <= n1): True

Ключі збережено: key_A.json, key_B.json
```

Використання примітивів RSA:

```
PS > python rsa.py

--------------------------------------------------
Приватний ключ (d, p, q) (hex):
(017a34636562d24b9077d6170437f774b6b950289afc95358ee75da730106c9fb2bf8078de146
e8744c5b2a42eda1baefe109360e94905e8fa326d3538204b98fb31,
3164dbcde91c53e8baec8810106859be7029cd530729257afe2e242daf7bb763ff,
0883f8086c409e3ae4bafbc1a4d81ff4671555dafcf3dd93cd563a0ca6f48e96e9)
Публічний ключ (n, e) (hex):
(01a49d563ea088da5e922444e8c2266a1a91be50af8756076dd8ee3af0e6f567ae7282db80b92
fde7849af2903a798927368bb5965fc917af0ebee549d9791f36d17, 010001)


Оригінальне повідомлення (hex):
cd9a7518b363a6b3a9b0d921d8b6c7b8a3db1cf39f9effa45f61548c0075dafd
Шифротекст (hex):
01a1efe368d64540fdfe8a82635414283944ff8a85769088bcd78dc76e7b1c02c87877235c94f0
05e92798c09f48081cec73a6fab5739ac30fa239b1b4fa99a05bf3
Розшифроване повідомлення (hex):
cd9a7518b363a6b3a9b0d921d8b6c7b8a3db1cf39f9effa45f61548c0075dafd
Підпис (hex):
019b175e6d89ac261eabf264225185e77687754701811be86f2db584513e005131b9a5458d42d0
28df4320b22256ad3dc13c05f72a8f03a74c5109b0e98f925547c5
Перевірка підпису:   True

--------------------------------------------------
Приватний ключ (d, p, q) (hex):
(782afd5ddc247c1072ca936431dadeacfde023efb2e077d59bddc9b0aa34ca63752c01033ccf6
0d08b466ff988708b5bd5b3c936dfa01f3e146c46a7149b29332249,
ad0ca3f8d759a9123d2b400e7a98f9b06e8f4797396b29985ffecf018c84ed9a6d,
ecc8dc4a4e04f7e8d4690893324444a1d3c63c8948cbbe7c553834d134e9fb9f0d3)
Публічний ключ (n, e) (hex):
(a00f6df2983d5aa52a9a0ca854f2daddadd9ff2d565ff2211ecdae1bb081b900878adf0e0bea3
```

```
1d8c5b3bae101c0c7b4b3c2a8a77b5e5677b611d7d1ca449c6177d7, 010001)


Оригінальне повідомлення (hex):
1c420651597097c252c1c7e72eb606c5430c324bd060d0620d54022ae573f9e1
Шифротекст (hex):
331c3d7fb3b6612d639013cac8b7d23a338073ec4282752f5d3f573320a6d39e4636d7c7dfc31c
20d0a637cca079db43c8895da7205f95b02474f59869d041ab3c0d
Розшифроване повідомлення (hex):
1c420651597097c252c1c7e72eb606c5430c324bd060d0620d54022ae573f9e1
Підпис (hex):
3bb53bc83c250f55dd145ac98929b9a7861bf79f1b9359866528ad188b87a861e45a592c9c8c2c
d784c23575abf077cad35aaa8188d58ce07e82e1f0ac239b3b1eb2
Перевірка підпису:   True
```

Відправка ключа

```
PS > python protocol.py
=== ПАРАМЕТРИ АБОНЕНТІВ ===

Публічний ключ абонента A (n, e):
(01a49d563ea088da5e922444e8c2266a1a91be50af8756076dd8ee3af0e6f567ae7282db80b92
fde7849af2903a798927368bb5965fc917af0ebee549d9791f36d17, 010001)

Приватний ключ абонента A (d, p, q):
(017a34636562d24b9077d6170437f774b6b950289afc95358ee75da730106c9fb2bf8078de146
e8744c5b2a42eda1baefe109360e94905e8fa326d3538204b98fb31,
3164dbcde91c53e8baec8810106859be7029cd530729257afe2e242daf7bb763ff,
0883f8086c409e3ae4bafbc1a4d81ff4671555dafcf3dd93cd563a0ca6f48e96e9)


Публічний ключ абонента B (n1, e1):
(a00f6df2983d5aa52a9a0ca854f2daddadd9ff2d565ff2211ecdae1bb081b900878adf0e0bea3
1d8c5b3bae101c0c7b4b3c2a8a77b5e5677b611d7d1ca449c6177d7, 010001)

Приватний ключ абонента B (d1, p1, q1):
(782afd5ddc247c1072ca936431dadeacfde023efb2e077d59bddc9b0aa34ca63752c01033ccf6
0d08b466ff988708b5bd5b3c936dfa01f3e146c46a7149b29332249,
ad0ca3f8d759a9123d2b400e7a98f9b06e8f4797396b29985ffecf018c84ed9a6d,
ecc8dc4a4e04f7e8d4690893324444a1d3c63c8948cbbe7c553834d134e9fb9f0d3)

Умова n <= n1 виконується: True

=== ЕТАП ВІДПРАВНИКА (Абонент A) ===
Випадковий секретний ключ k (hex):
fb31feb984a9e1627e01ce60edde28293d326b599c184a2c42a3ba9c760e857ee1a4fcac409272
```

```
b3fad0c60b739186e5c1f5cb8708f171445ee20bf911d887bc83
Підпис S = k^d mod n (hex):
94bbcc9c77004a69fc6aaac4a795f44095f9f4d40f35cf4251f3089ecafebc1a68ecf02bb75b3e
86c0cfd87bbb244885388e7837f431a8ded1ca5dabe5a7c754aa
Криптограма ключа k1 = k^e1 mod n1 (hex):
3ca04876c28b87b1cea34e907be4a9000331400190b61737532388088b1648ebb3e3db9029d763
d8c6ce266c6810be3c1ab0db9f54d2d48867119f2bd668cf88847f
Криптограма підпису S1 = S^e1 mod n1 (hex):
0bacffad88a4aa3690eaccd3f50c9e885d12e5a999a61d29c912c11b3764241b4898498c77ee8e
f7fb68f6f3bc187cf4b06bb6dce775476b94b548e9a331c19f8e3b
Справжність відправника: підпис S створено секретним ключем d абонента A.
Конфіденційність: k і S зашифровано відкритим ключем (e1, n1) абонента B.

=== ЕТАП ОТРИМУВАЧА (Абонент B) ===
Розшифрований ключ k = k1^d1 mod n1 (hex):
fb31feb984a9e1627e01ce60edde28293d326b599c184a2c42a3ba9c760e857ee1a4fcac409272
b3fad0c60b739186e5c1f5cb8708f171445ee20bf911d887bc83
Розшифрований підпис S = S1^d1 mod n1 (hex):
94bbcc9c77004a69fc6aaac4a795f44095f9f4d40f35cf4251f3089ecafebc1a68ecf02bb75b3e
86c0cfd87bbb244885388e7837f431a8ded1ca5dabe5a7c754aa
Перевірка підпису S^e mod n (hex):
fb31feb984a9e1627e01ce60edde28293d326b599c184a2c42a3ba9c760e857ee1a4fcac409272
b3fad0c60b739186e5c1f5cb8708f171445ee20bf911d887bc83
Результат перевірки справжності (k == S^e):  True
Конфіденційність: розшифрувати k1, S1 може лише абонент B – власник секретного
ключа d1.
Справжність відправника: підпис перевірено відкритим ключем (e, n) абонента A.

=== ПІДСУМОК РОБОТИ ПРОТОКОЛУ ===
Початковий ключ k == Отриманий ключ k: True
Підпис відправника A підтверджено:     True
```

---

## Перевірка взаємодії реалізації з тестовим середовищем

Тестове середовище: http://asymcryptwebservice.appspot.com/?section=rsa. Усі числа нижче наведено в hex і скорочено; повні значення s відповіді сайту - у файлі `interop_log.txt`. На сайті відкритий показник вводився як `10001` (це те саме, що `010001` у виводі програми, тобто 65537).

### Як працює тестове середовище

- Розділ **Server Key** генерує ключову пару *сервера* певної довжини (e = `10001`) і повертає відкритий ключ (Modulus, Public exponent). Секретний ключ залишається на

сервері в межах сесії.

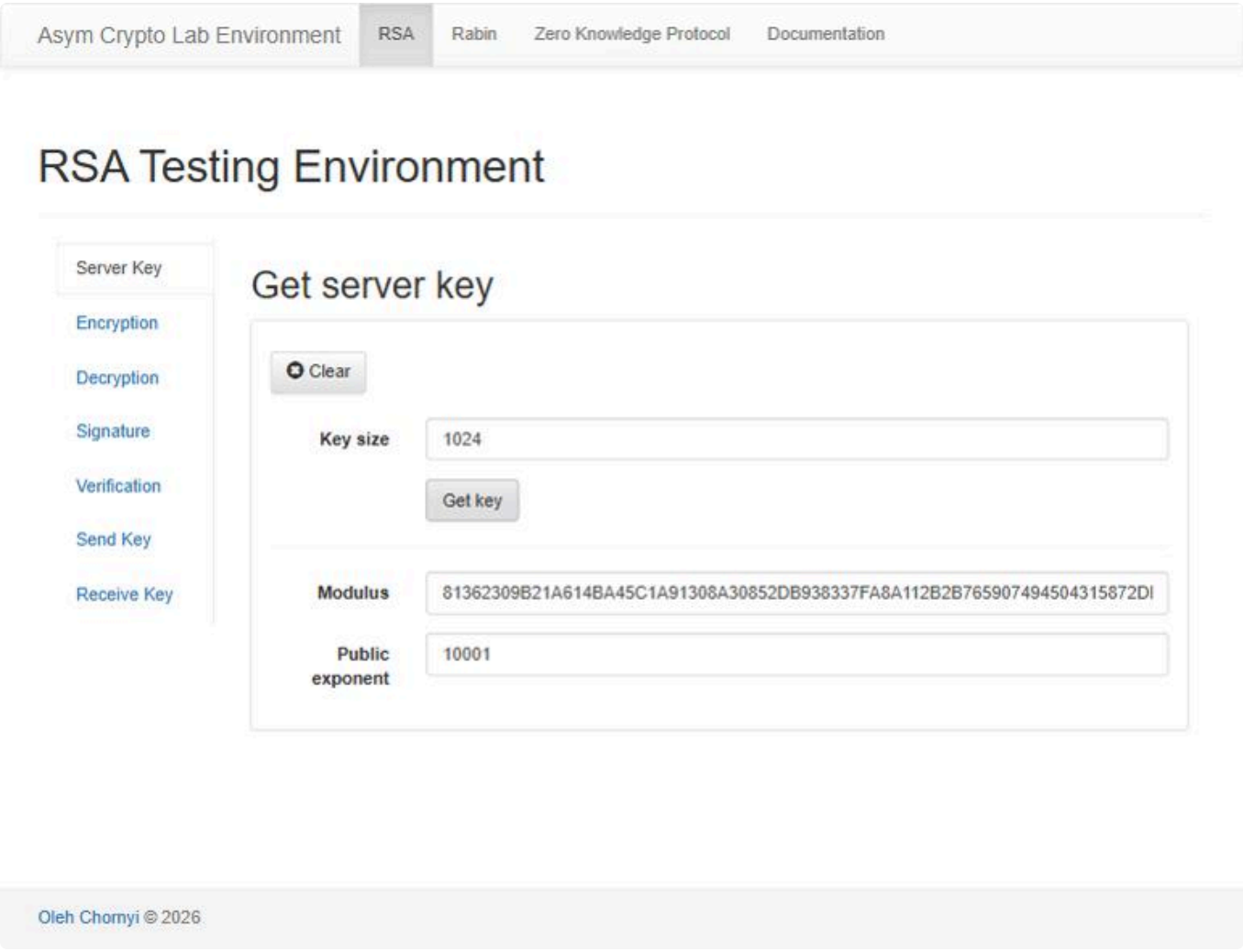
- Розділи **Decryption**, **Signature** і **Receive Key** працюють із *секретним ключем сервера*, тому власним ключем у них нічого не розшифрувати і не підписати. Розділи **Encryption**, **Verification** і **Send Key** приймають відкритий ключ (Modulus, Public exponent) від користувача - саме в них вводяться ключі наших абонентів A і B.
- Це класичний RSA із методичних вказівок: `C = M^e mod n`, `S = M^d mod n`. Тип *Bytes* - hex-байти як ціле число, *Text* - те саме для UTF-8 байтів тексту. Сайт повертає hex великими літерами без ведучих нулів, тому `hex_to_int` у коді приймає будь-який регістр і непарну кількість цифр.
- Протокол розсилання ключів на сайті збігається з методичкою: `S = k^d mod n`, `k1 = k^e1 mod n1`, `S1 = S^e1 mod n1`; обов'язкова умова `n1 ≥ n`.

## Схема перевірки

Згідно з методичкою кожну операцію перевірено в обидва боки: **(а)** власна реалізація → сервер; **(б)** сервер → власна реалізація. Значення переносились вручну (копіюванням), без мережевих запитів з коду; для цього написано `site_interop.py` (команди `encrypt`, `decrypt`, `sign`, `verify`, `send-key`, `receive-key`).

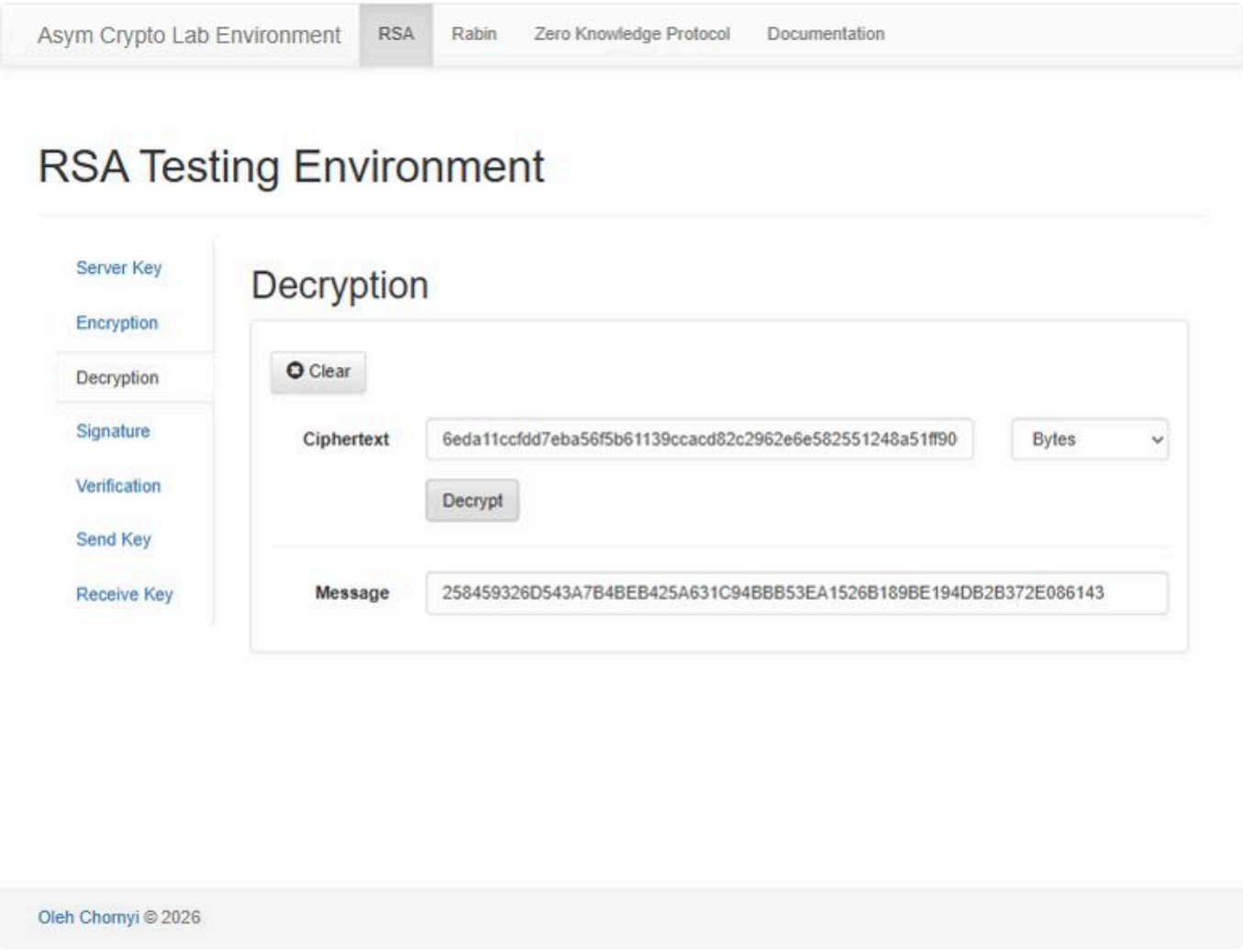
Ключі абонентів: A - n має 521 біт, B - n1 має 528 біт (n ≤ n1). Розмір ключа сервера підбирався під умову протоколу `n1 ≥ n`: **1024 біт** для A → сервер і **512 біт** для сервер → B.

Відкритий ключ сервера (1024 біт):

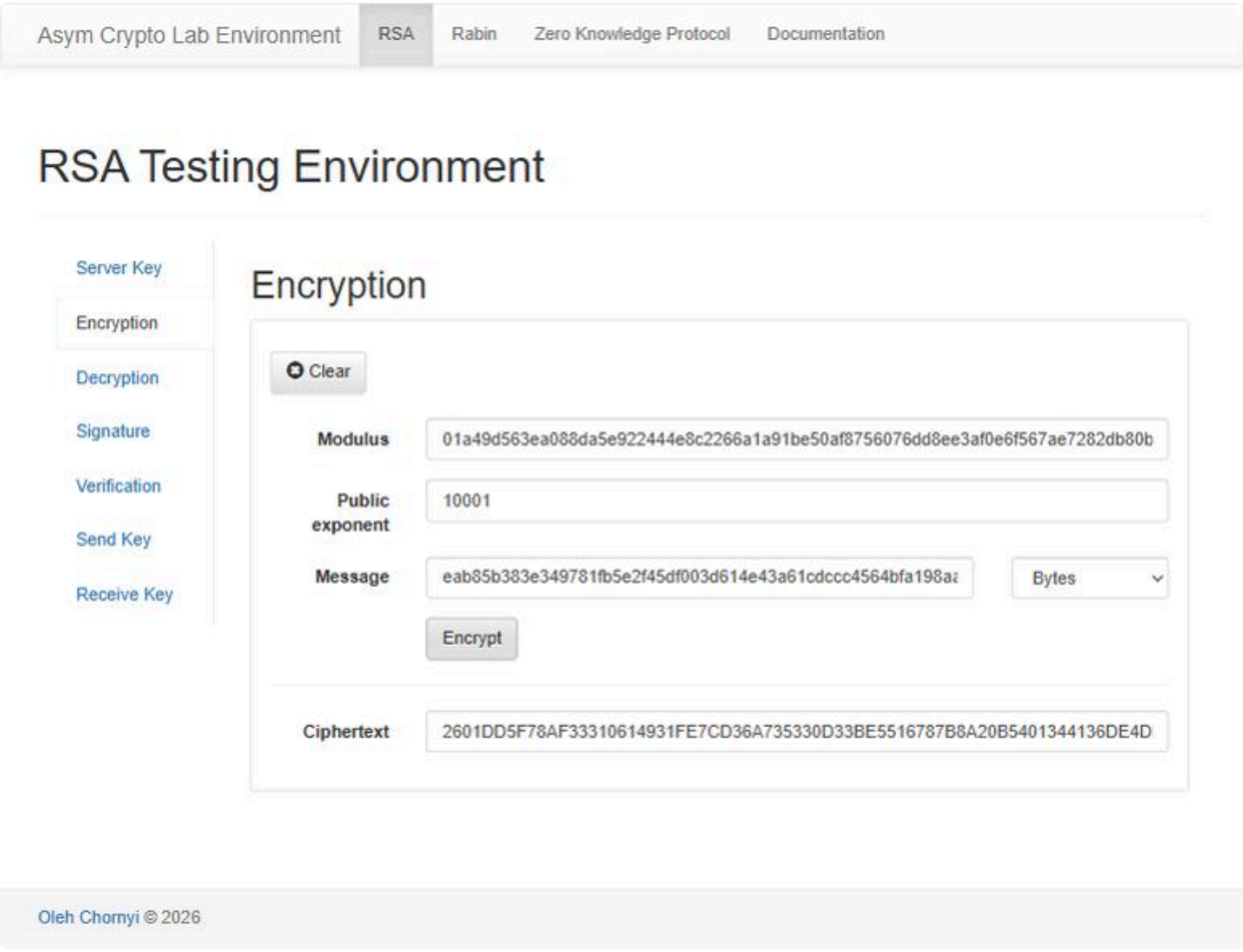



## Шифрування і розшифрування

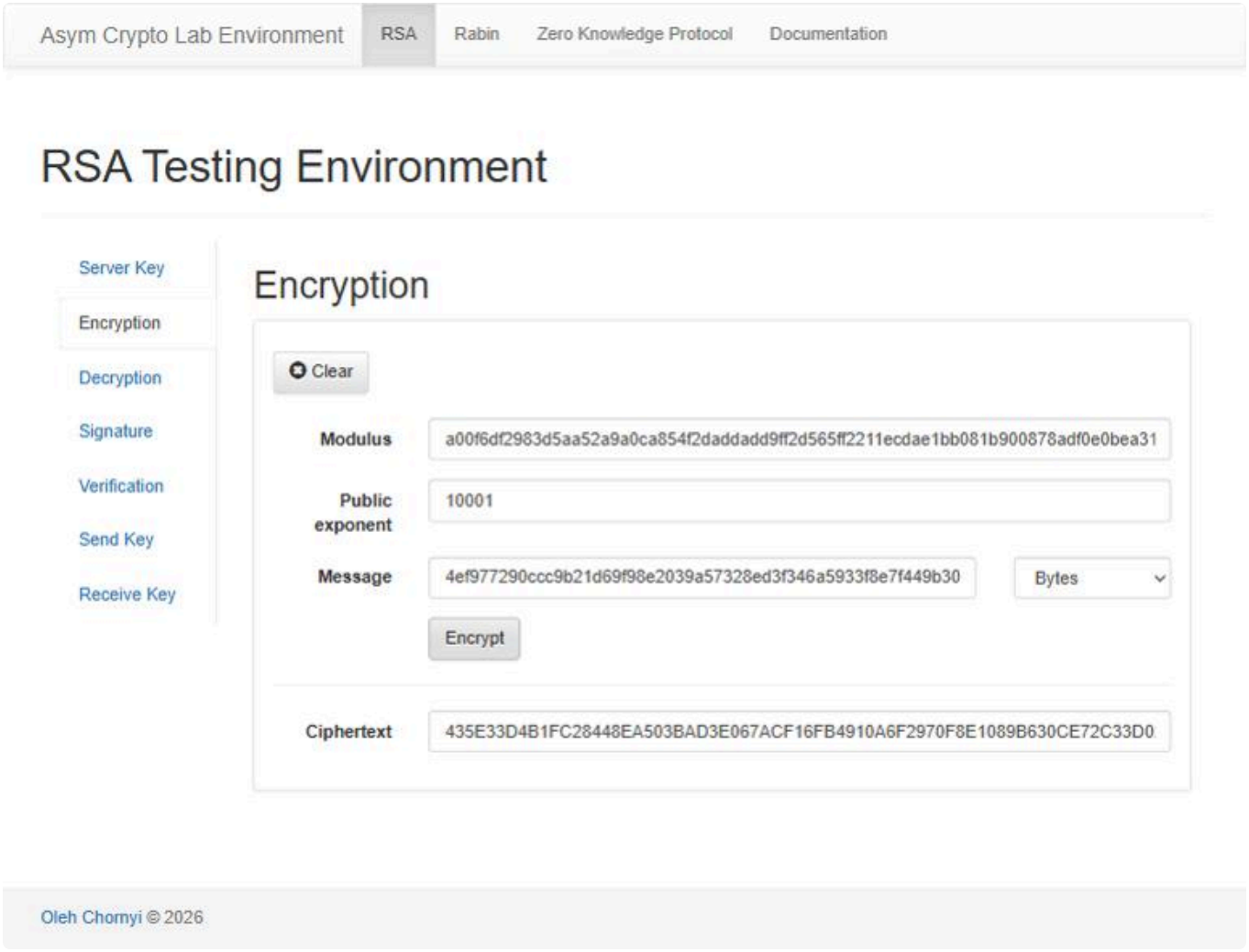
**(a) Наш `Encrypt` → розшифрування на сервері.** Повідомлення `M = 258459326d54…2e086143` зашифровано відкритим ключем сервера: `C = 6eda11ccfdd7…af6babbb`. У розділі Decryption (тип Bytes) сервер повернув `258459326d54…2e086143` — збігається з `M`.



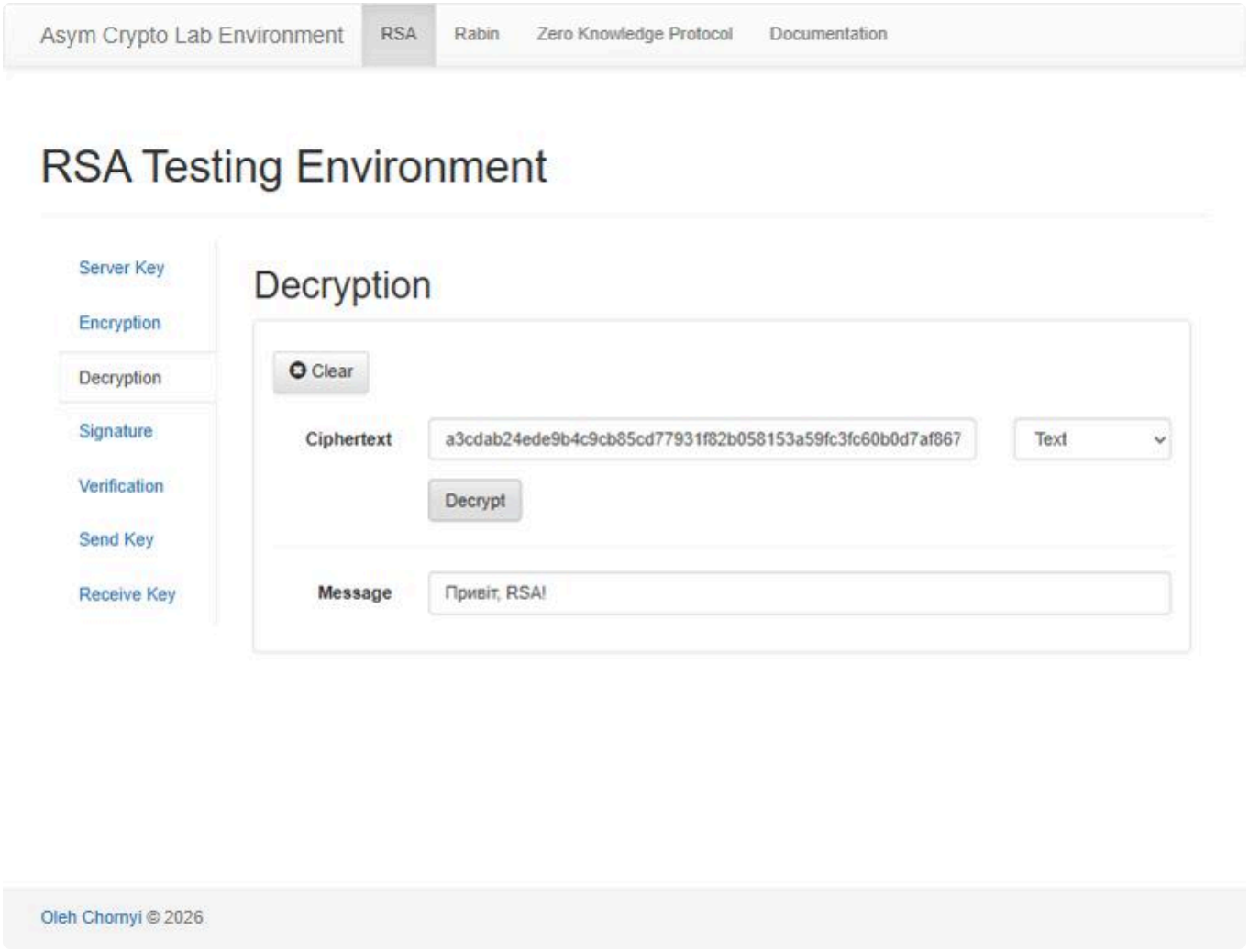

**(б) Шифрування на сервері → наш `Decrypt`.** У розділі Encryption введено відкритий ключ абонента A та `M = eab85b383e34…df822d24`; сервер повернув `C = 2601DD5F78AF…9FC1A6BE`. Наш `Decrypt` секретним ключем A відновив `M = eab85b383e34…df822d24`.



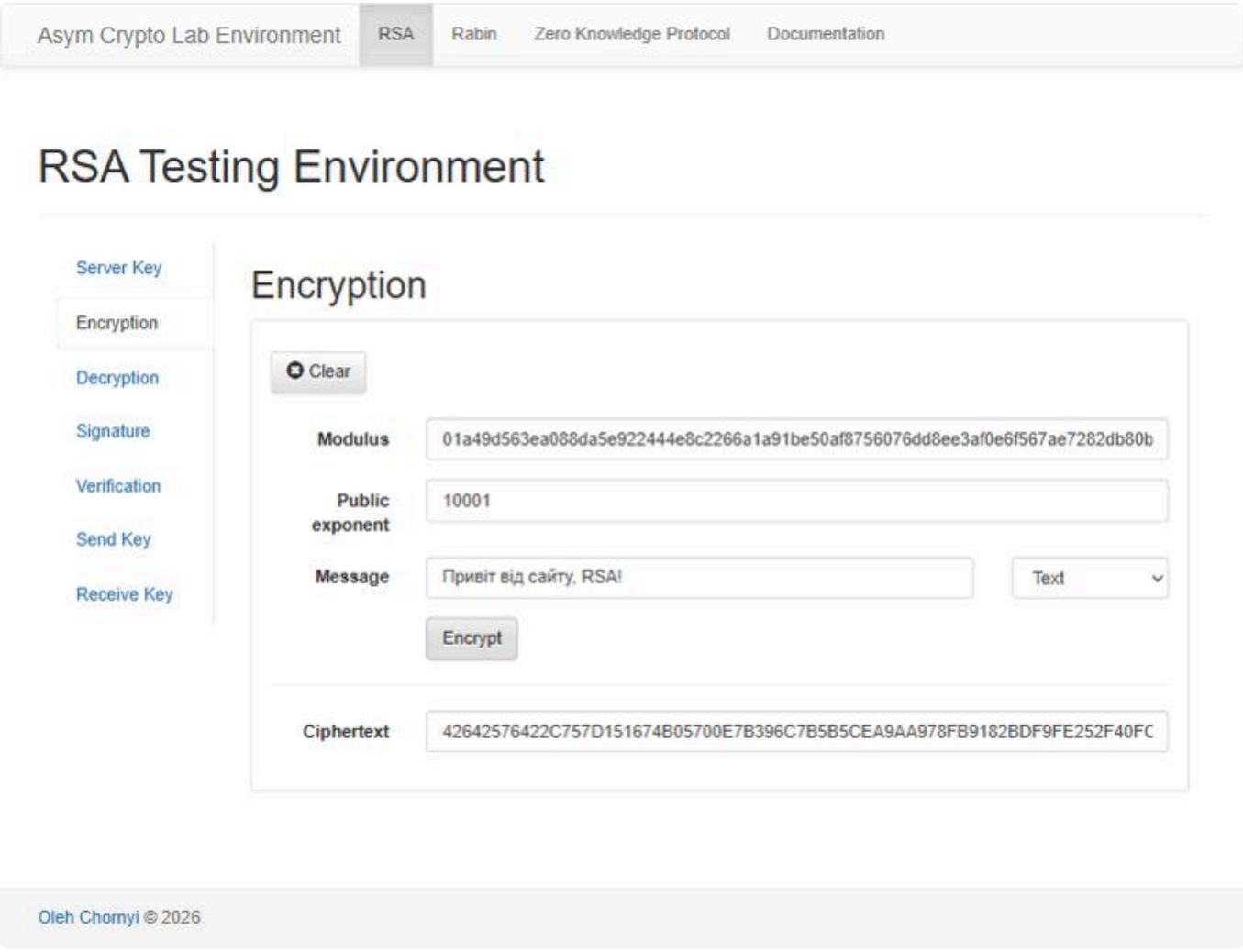

Те саме для абонента B: `M = 4ef977290ccc…c7c9c3c2`, `C = 435E33D4B1FC…12C3ECCE`; `Decrypt` секретним ключем B повернув `M`.




**Текстові повідомлення (тип Text).** Наш `Encrypt` для сервера (ключ 512 біт), `M = "Привіт, RSA!"` (UTF-8: `d09fd180d0b8d0b2d196d1822c2052534121`), `C = a3cdab24ede9…b59360bf`; у розділі Decryption з типом Text сервер повернув `Привіт, RSA!`.

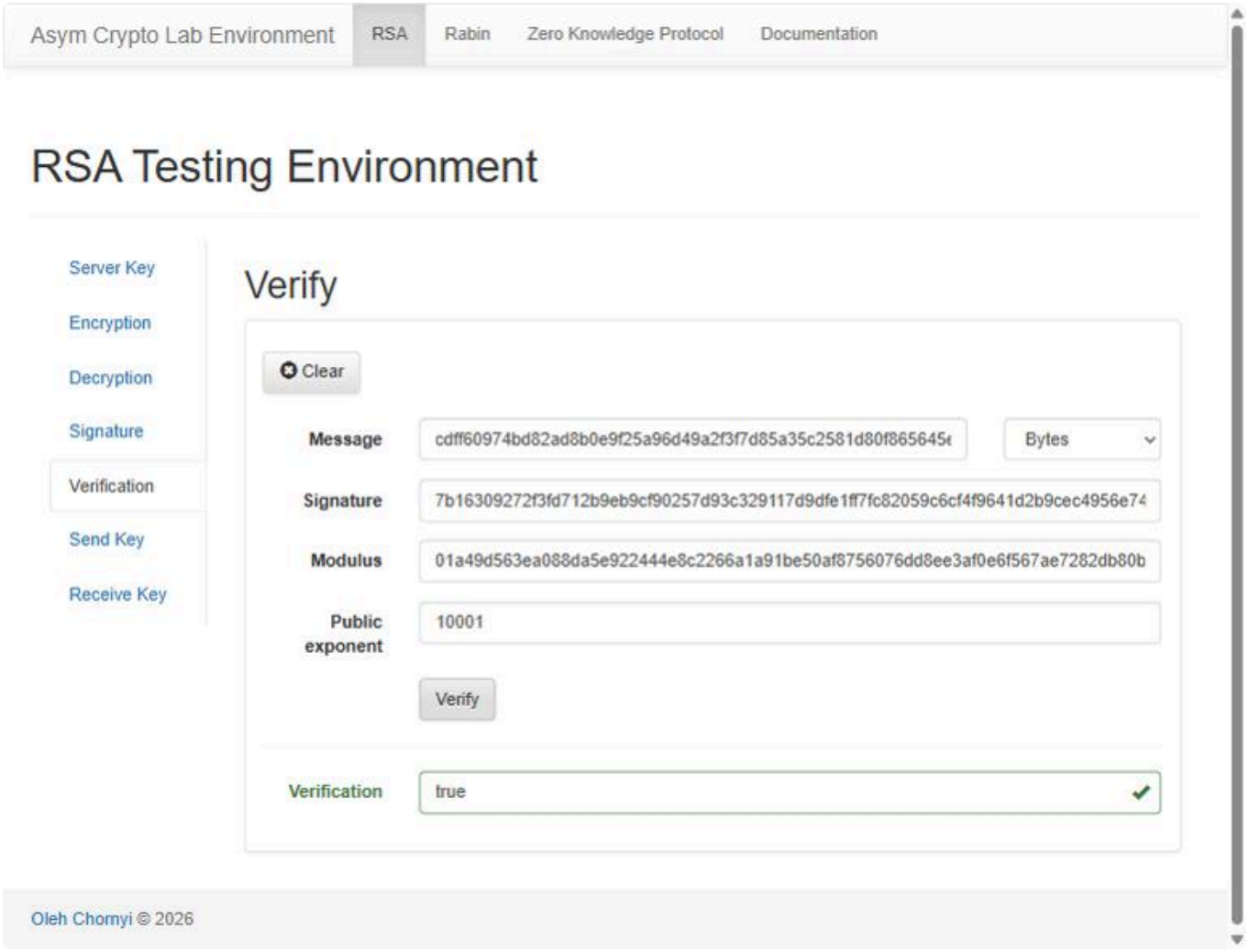



На сайті (Encryption, тип Text, ключ A) зашифровано `Привіт від сайту, RSA!`: `C = 42642576422C…FC1FDA94`; наш `Decrypt` повернув той самий текст.

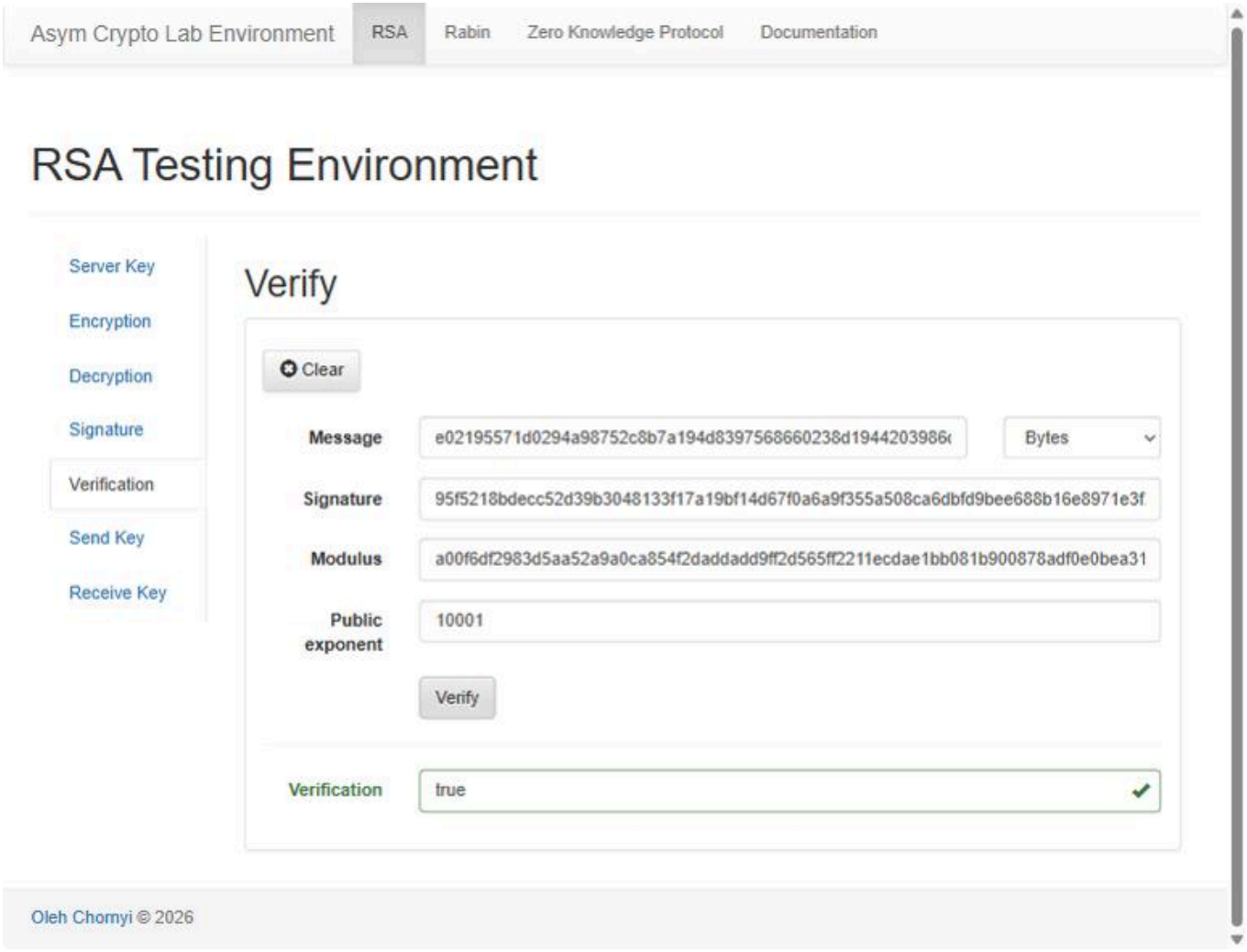



## Цифровий підпис

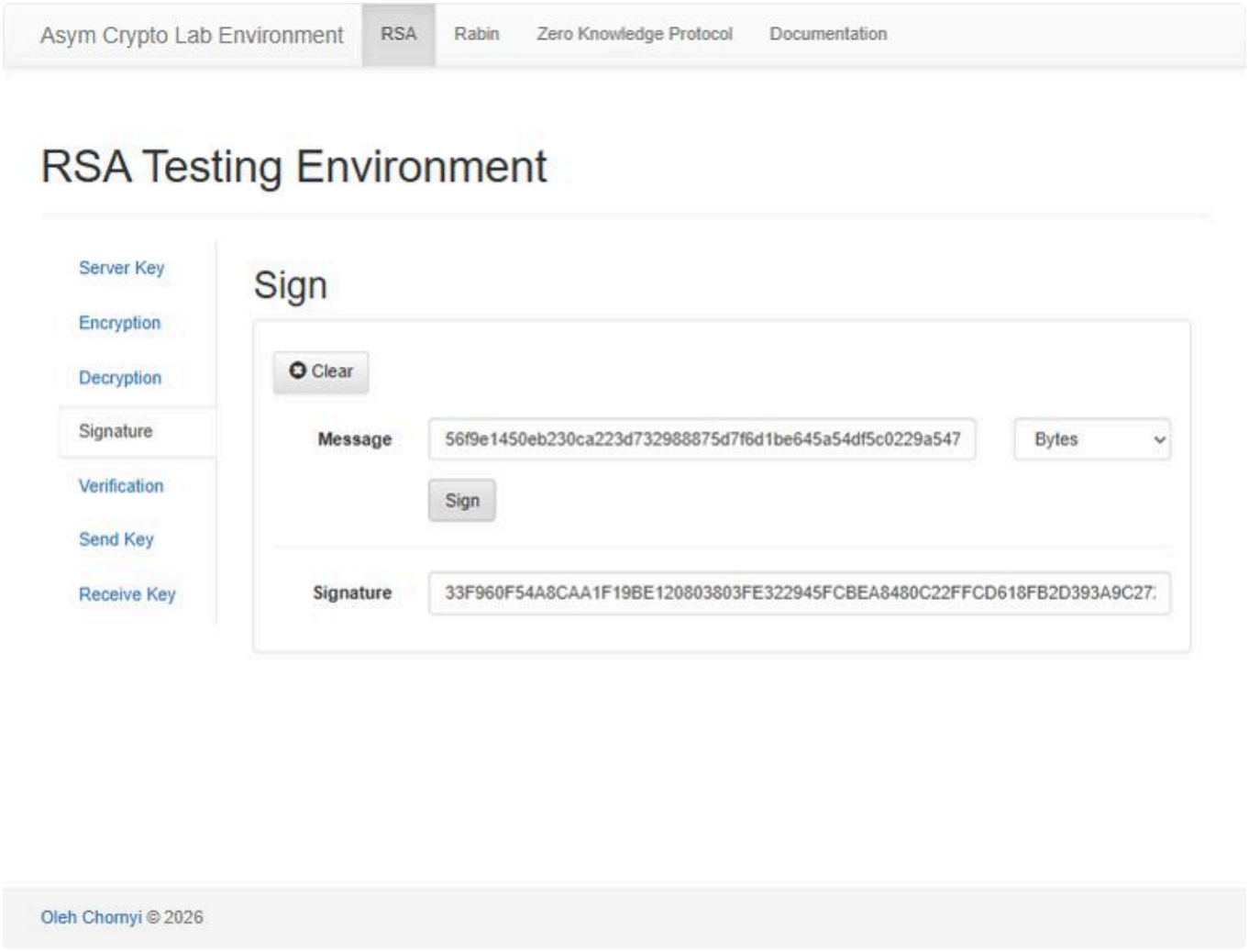
**(a) Наш `Sign` → перевірка на сервері.** Абонент A: `M = cdff60974bd8…e307d360`, `S = 7b16309272f3…f26addd1`. У розділі Verification введено `M`, `S` та відкритий ключ A — результат **true**.




Абонент B: `M = e02195571d02…e91d048d`, `S = 95f5218bdecc…fe5db9bd`; Verification — **true**.

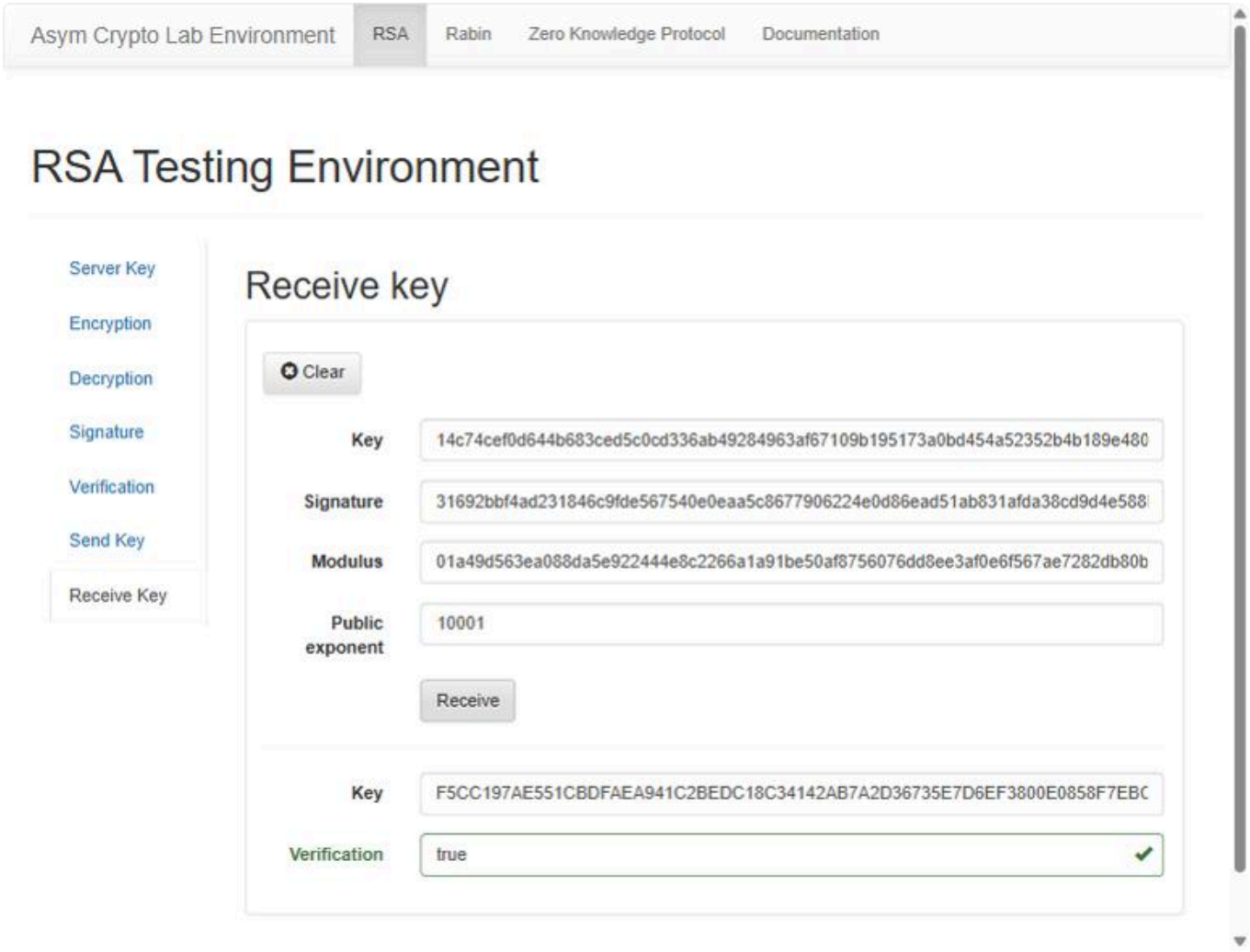



**(б) Підпис на сервері → наш `Verify`.** У розділі Signature сервер підписав `M = 56f9e1450eb2…04e676c4` і повернув `S = 33f960f54a8c…f2aa6aac`. Наш `Verify` відкритим ключем сервера (1024 біт) повернув **True**.

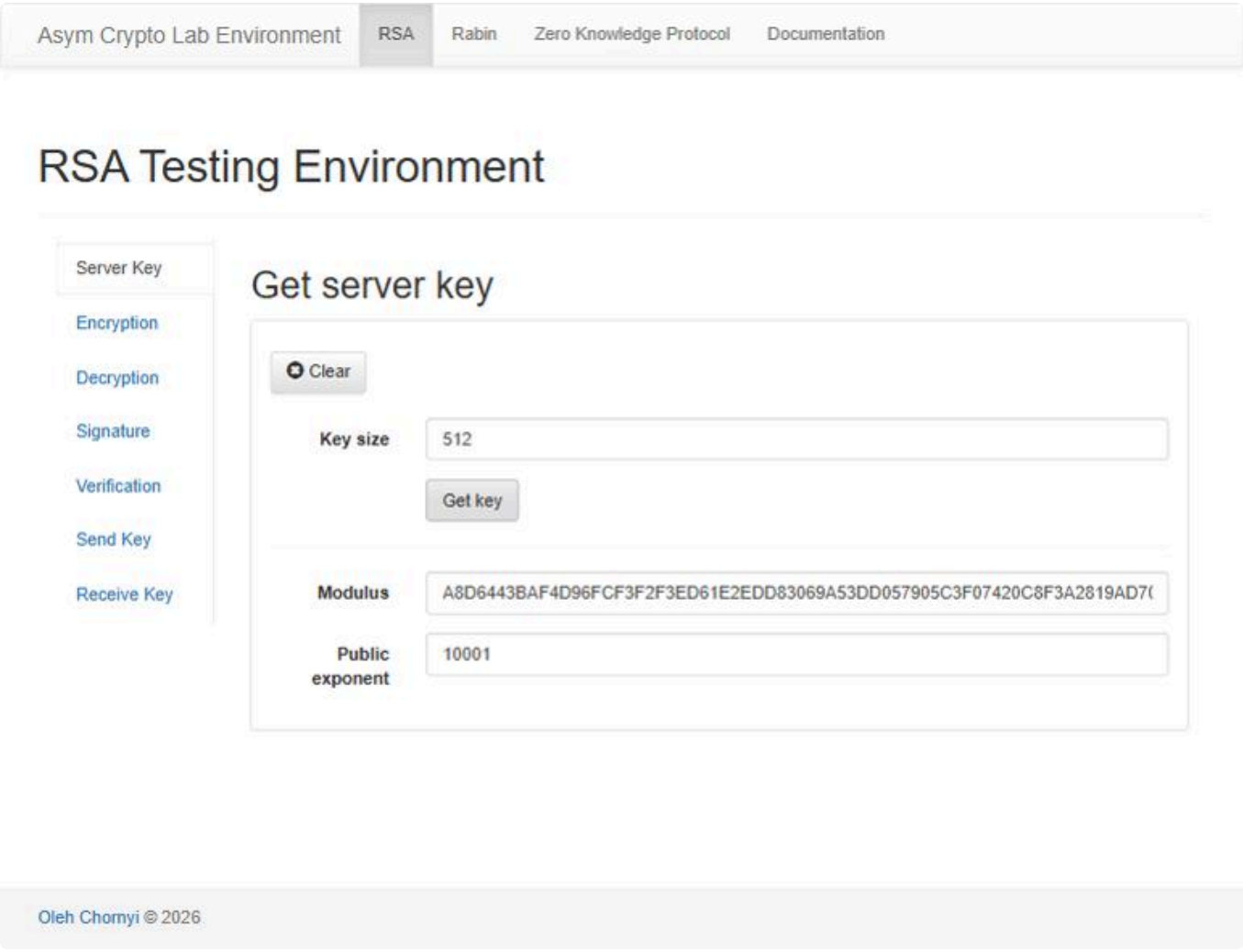



## Протокол конфіденційного розсилання ключів

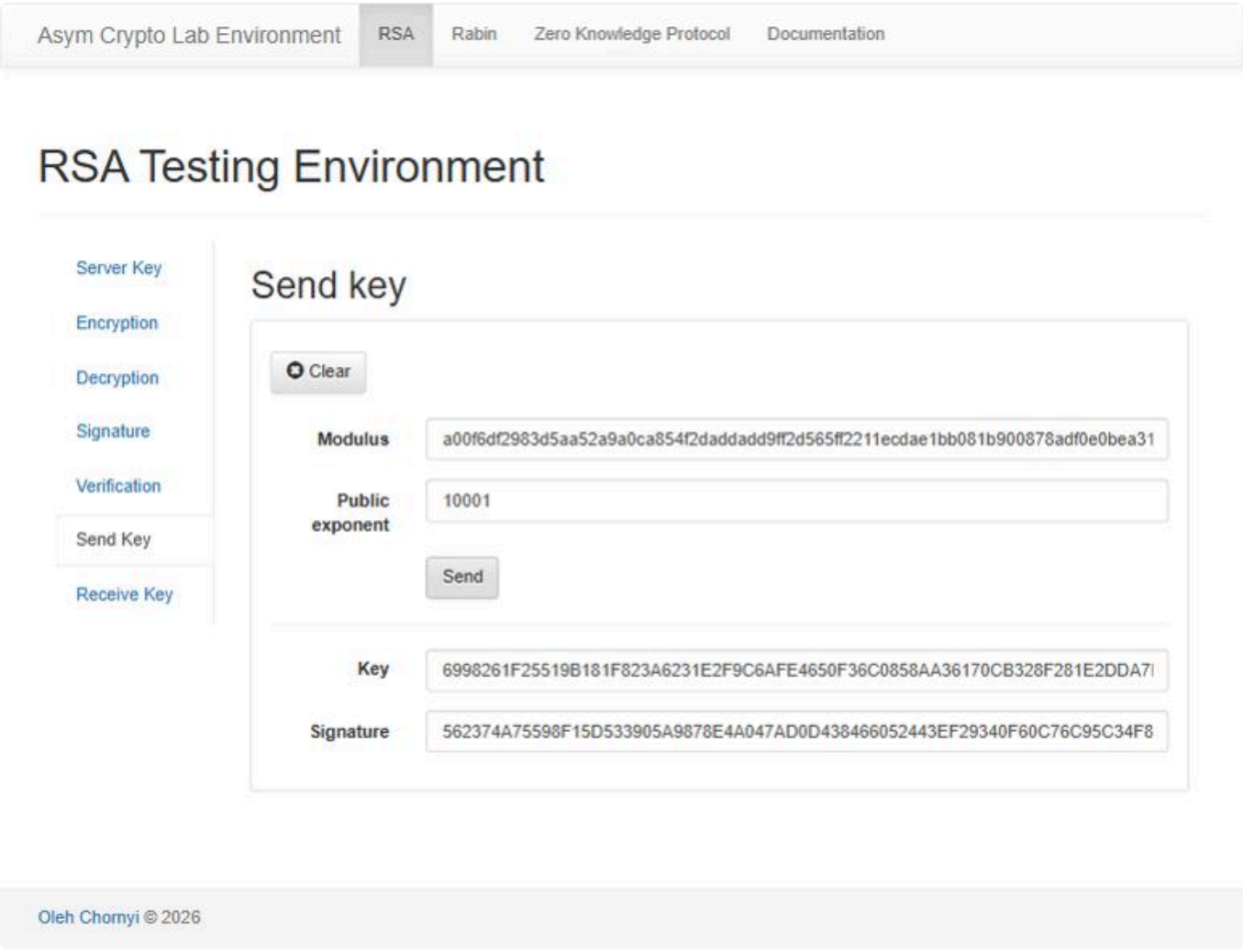
**(a) Наш `SendKey` (A → сервер).** Ключ сервера — 1024 біт ( `n1 ≥ n` ). Випадковий `k = f5cc197ae551…87851461` ; `S = k^d mod n = 8431d4376f93…25413aff` ; надіслано `k1 = 14c74cef0d64…e3de11f8` , `S1 = 31692bbf4ad2…05d90e49` . У розділі Receive Key введено `k1` , `S1` та відкритий ключ A (відправника): сервер повернув `Key = f5cc197ae551…87851461` (збігається з `k` ) і **Verification = true**.




**(б) `Send Key` на сервері → наш `ReceiveKey` (сервер → B).** Для умови `n1 ≥ n` згенеровано новий ключ сервера на 512 біт:




У розділі Send Key введено відкритий ключ B; сервер повернув `k1 = 6998261F2551…3CE49BFC` і `S1 = 562374A75598…69BA0F55`.




Наш `ReceiveKey` секретним ключем B розшифрував пакет, отримав `k = 10ff13a6c6c86808` і перевірив підпис відкритим ключем сервера: `k == S^e mod n` — **True**.

Конфіденційність забезпечується шифруванням `k` і `S` відкритим ключем одержувача `(e1, n1)` — розшифрувати може лише власник `d1`. Справжність відправника забезпечується підписом `S = k^d mod n` на секретному ключі відправника, який перевіряється його відкритим ключем `(e, n)`.

## Підсумок перевірок

| № | Перевірка | Результат | Скріншот |
|---|---|---|---|
| 1 | `Encrypt` (ключ сервера) → Decryption на сайті | `M` збігся | 02 |
| 2 | Encryption на сайті (ключ A) → `Decrypt` | `M` збігся | 03 |
| 3 | Encryption на сайті (ключ B) → `Decrypt` | `M` збігся | 04 |
| 4 | `Sign` (A) → Verification на сайті | true | 05 |
| 5 | `Sign` (B) → Verification на сайті | true | 06 |

| № | Перевірка | Результат | Скріншот |
|---|---|---|---|
| 6 | Signature на сайті → `Verify` | True | 07 |
| 7 | `SendKey` (A → сервер) → Receive Key на сайті | `Key = k`, true | 08 |
| 8 | Send Key на сайті → `ReceiveKey` (B) | `k` отримано, True | 10 |
| 9 | Text: `Encrypt` → Decryption на сайті | `Привіт, RSA!` | 11 |
| 10 | Text: Encryption на сайті → `Decrypt` | `Привіт від сайту, RSA!` | 12 |

---

# Контрольні питання

1. Для генерації пари простих чисел для кожного кандидата виконується тест Міллера-Рабіна і потім з цих потенційно криптографічно нестійких простий чисел генеруються числа виду $p = 2ip' + 1$, що потім знову проходять тест Міллера-Рабіна, якщо результат позитивний, то число може бути використано у криптосистемі RSA.
2. Головними вимогами до ключів є наступні: $gcd(e, \phi(n)) = 1$ та $ed \equiv 1 \mod \phi(n)$ . У коді це реалізовано прямолінійно `if math.gcd(e, phi_n) != 1:` та `if d * e % phi_n != 1:` відповідно
3. У `Encrypt()` потрібно передавати публічний ключ та відкритий текст
   У `Decrypt()` потрібно передавати приватний ключа та шифротекст
   У `Sign()` потрібно передати приватний ключ підписувача та повідомлення
   У `Verify()` потрібно передати публічний ключ підписувача, повідомлення та цифровий підпис
   Загалом всі ці функції зводяться до одного принципу роботи RSA. Функція шифрування $C = M^e \mod n$ , відповідно використовує тільки дані публічного ключа $(e, n)$. Тоді як функція розшифрування $M = C^d \mod n$, де використовуються тільки дані приватного ключа $(d, p, q)$, $n = pq$. "Під капотом" функції підпису та його перевірки працюють з використанням шифрування та розшифрування, тільки публічний та приватний ключ міняються місцями - підписується (шифрується) повідомлення приватним ключем, а перевіряється (розшифровується) публічним.
   Перевіряється, щоб $M_{factual} = (M_{orig}^d)^e \mod n$
4. Відправник створює цифровий підпис ключа (автентифікація), а потім шифрує його відкритим ключем отримувача (конфіденційність).
   Отримувач розшифровує отриману криптограму свої приватним ключем (конфіденційність), на цьому моменті він має ключ та його підпис. Далі отримувач перевіряє підпис за допомогою публічного ключа відправника (автентифікація)

5. Отримувач розшифровує підпис публічним ключем відправника і порівнює його значення із отриманим ключем, якщо співпадає, то ключ вважається автентичним

---

# Висновки

У ході цієї лабораторної роботи ми ознайомилися із тестами перевірки чисел на простоту і методами генерації ключів для асиметричної криптосистеми типу RSA. На практиці ознайомилися із системою захисту інформації на основі криптосхеми RSA, організували з її використанням системи засекреченого зв'язку і електронний підпис. Вивчили протоколу розсилання ключів на основі RSA.